

Kasa bez zarządu

A savings and loan fund with **no board and no treasurer**. Members save together and lend each other money at 0%. The rules are a Solana program with no admin key.

0

people who approve a loan

100%

of every loan covered by locked savings

anyone

can collect an overdue installment

THE PROBLEM

A group's money only works if everyone trusts the treasurer

Who we build for: small groups in Poland that already pool money. Workplace *kasy zapomogowo-pożyczkowe* (KZP) in schools, offices and factories, teams with a "kasa koleżeńska", friends and families lending to each other.

Members save every month and borrow at 0%, with co-members vouching for the loan. It works only through intermediaries:

- **the treasurer** holds the money and the books,
- **the board** decides who gets a loan,
- **the employer's payroll** deducts installments and is the only real enforcement.

When that trust fails

A PKZP treasurer in Jawor county took **61,000 zł** of colleagues' savings and lost it on a crypto exchange.

| Puls Legnicy, March 2024

A PKZP cashier hid **261,510 zł** of withdrawals with falsified cash reports for **ten years**.

| Court of Appeal in Katowice, II AKa 98/15

Informal groups have it worse: "the kasa" is simply someone's private bank account.

THE REDESIGN

Each intermediary becomes a rule in the program

ROLE	KZP / GROUP KASA TODAY	KASA BEZ ZARZĄDU
Custody	Treasurer's or KZP bank account	Vault = token account owned by the program; only the rules move tokens out
Loan approval	Board decision	Nobody: pays out when locked savings (own + guarantors') cover 100%
Enforcement	Payroll deduction, board chasing debtors	After due date + grace, anyone triggers collection from collateral
Your savings back	When the board agrees, usually when leaving	Any time, for unlocked savings
Changing rules	General meeting or board	No instruction changes them; no admin key
Who can lose	Every member, if the books fail	Only a guarantor who chose to vouch, capped at the pledge

HOW IT WORKS

Save, borrow with guarantors, repay or be collected

1 · SAVE

Deposit into the vault

Anyone joins, no approval. Free savings are withdrawable any time.

2 · BORROW

Up to 2× savings

Own free savings lock first. The rest must be pledged by co-members.

3 · PAY OUT

100% covered → paid

The borrower pays the loan out to their wallet. Nobody signs off.

4 · REPAY

Installments at 0%

Each repayment unlocks the same amount of collateral, guarantors first.

Bartek borrows 1000 zł in 4 installments



His own savings 500 zł · Anna pledges 300 zł · Celina pledges 200 zł → collateral 1000 zł = loan, so he pays it out.

He repays once, then stops

The repayment frees 250 zł of the guarantors' pledges (3:2). After installment 2 + grace, **Celina presses "Egzekwuj"**: 250 zł comes from Bartek's locked savings. Later installments take his last 250 zł, then 250 zł from Anna and Celina pro rata.

The fund ends whole

Bartek: kept the cash, lost 500 zł of savings. Anna -150 zł, Celina -100 zł: the risk they chose.

Fund: 0 zł lost. Everyone withdraws the rest; vault ends at 0 zł. All on devnet (docs/devnet-e2e.md).

An overdue installment is collected by anyone, with nobody's consent

Bartek pożycza 1 000 zł

Splacana

4 raty po ok. 250 zł · co 1 min · [konto pożyczki](#)

Zabezpieczenie pozostałej kwoty: 750 zł z 750 zł

Bartek (własne): 500 zł Anna poręcza: 150 zł Celina poręcza: 100 zł

Splacone: 250 zł Zostało: 750 zł

Rata 1: 250 zł
splacona

Rata 2: 250 zł
zaległa – do egzekucji

Rata 3: 250 zł
termin za 45 s

Rata 4: 250 zł
termin za 1 min 45 s

⚠ Zaległe 250 zł. Program pozwala **każdemu** ściągnąć tę kwotę z zabezpieczeń: najpierw z oszczędności pożyczkobiorcy (Bartek), potem proporcjonalnie z poręczeń. Bez zarządu, windykacji i niczyjej zgody.

Egzekwuj zaległą ratę

Spląć za pożyczkobiorcę (zł)

250

Spląć

```
// instructions/repay.rs: no signer required
pub fn handle_collect_overdue(ctx) {
    let overdue = loan.overdue(now,
        kasa.period_secs, kasa.grace_secs)?;
    require!(overdue > 0, NothingOverdue);
    check_guarantors(&loan, &guarantors)?;
    // borrower's savings first,
    // then guarantors pro rata
    seize(&mut loan, &mut borrower,
        &mut guarantors, overdue)?;
}
```

The tokens are already in the vault, so collection is bookkeeping on locked savings: it needs no signature from the borrower, the guarantors or us. It replaces the payroll deduction and the board's debt collection.

Two invariants, tested after every step

1. Every loan is fully covered

Pay-out requires locked savings = loan amount.
Repayments release collateral; collection consumes it.
Collateral always equals what is still owed.

2. The vault holds every free saving

$$\text{vault} = \Sigma \text{ savings} - \Sigma \text{ lent}$$

So anyone can withdraw what is theirs at any moment. A bank run cannot fail: the tests end with everyone withdrawing everything and the vault at 0.

The only risk is a chosen one

Only a guarantor who vouched for someone who didn't pay can lose, and at most the amount they pledged. That is the risk a KZP *poręczyciel* takes today, now explicit and visible to everyone.

QUESTIONS YOU WILL ASK

Someone disappears? Who can do what? Can we change it?

WHO GOES SILENT	WHAT HAPPENS
Borrower	Installments collected from their locked savings, then guarantors'
Guarantor	Nothing to do: the pledge unlocks as the loan is repaid
Founder	Nothing: no rights after creation
Us / the website	Nothing: program + IDL on-chain, any client works

ACTION	WHO
Join, deposit, withdraw free savings	Anyone, for themselves
Borrow / guarantee	Members, within the kasa's limits
Pay out / cancel a loan	Only the borrower
Repay, collect overdue	Anyone
Change rules, move others' savings	No instruction allows it, for anyone

Honest caveat: on devnet our deploy key can still upgrade the program code. Kept for bug fixes during the hackathon; next step: a members' multisig, or remove it with `--final`.

Every claim is backed by a test or a transaction

12/12

Rust tests: LiteSVM with the real SPL Token program, every rule-breaking attempt rejected

6/6

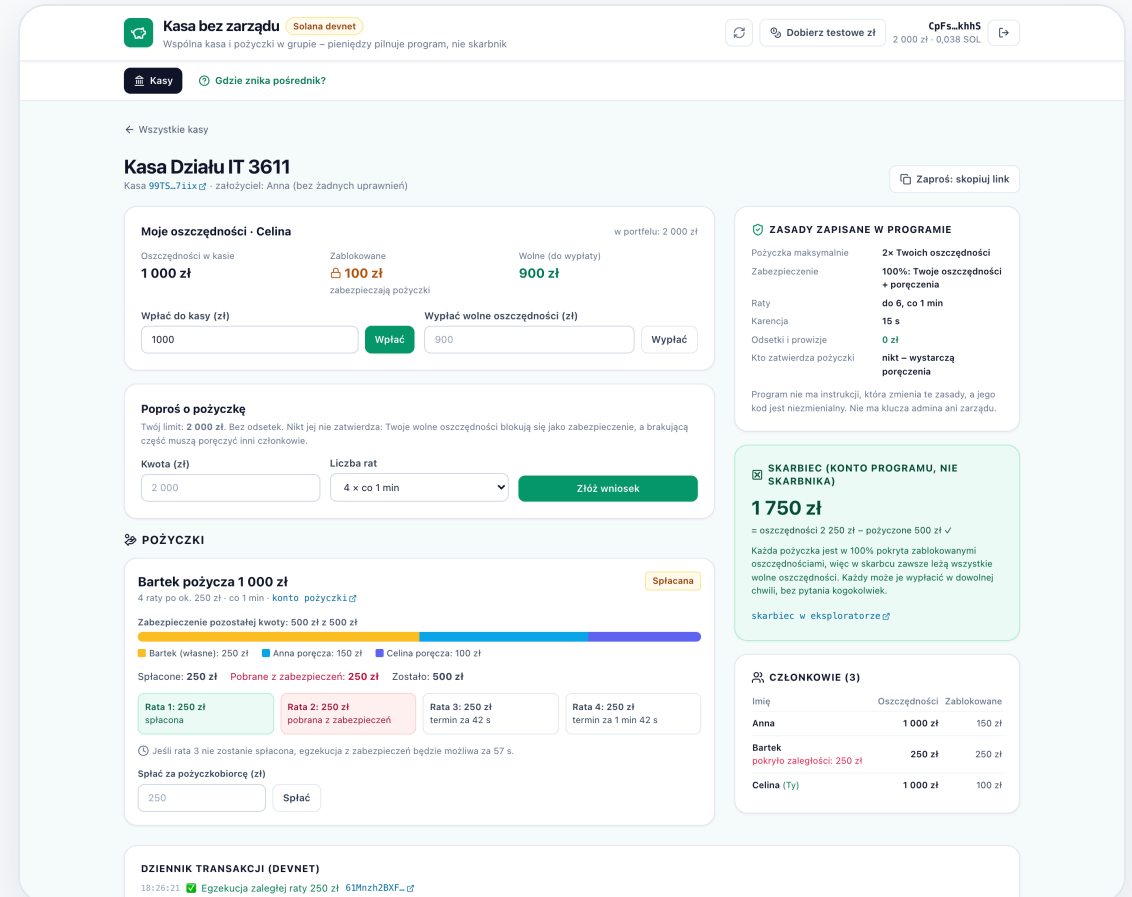
UI math tests: the app's schedule matches the program's

E2E

Devnet run with explorer links: loan, 2 guarantors, 2 collections, bank run → vault 0

UI

Playwright drives 3 wallets through the deployed app



Why blockchain, and why no arbiter

Why not a database?

In a database the intermediary is whoever runs it: they can edit a balance, hold back a withdrawal or quietly "borrow" from the pot, which is how the cases on slide 2 happened. Here the balance is a program-owned account, the rules are public code no instruction can bypass, and collection needs no trusted server. Solana makes it cheap enough for a 100 zł monthly contribution.

Why not an escrow marketplace?

We started there and dropped it. For physical goods someone must judge delivery: an arbiter (still an intermediary) or bonds that punish honest parties too. So we chose a relationship where **the program sees every condition it enforces**: deposits, balances, the schedule, the clock. Lending, the pattern Superteam calls the best example, in a form Polish workplaces already use.

What it doesn't do yet, and what we'd build next

Limitations

- Test złoty from a devnet faucet; the program takes any SPL mint (production: a stablecoin)
- Members need a Solana wallet
- Collection needs someone (or a bot) to send the transaction
- Contributions are voluntary, savings earn no interest (as in a KZP)
- The 2021 Act still expects a board; legal recognition needed
- Program still upgradeable by our deploy key
- Not audited, devnet only

With another week

- Standing orders without a bank: SPL delegation pulls installments and contributions
- *Zapomogi*: grants released by member vote
- PLN stablecoin, BLIK on-ramp, passkey wallets
- Public collector bot and due-date reminders
- Upgrade key to a members' multisig, audit, mainnet

Try it: web-production-ad49f.up.railway.app · **Code:** github.com/jxroo/kasa-bez-zarzadu